

HPC Lab4.5 Report

- 1 实验目标 2
- 2 FP64 模拟原理与 Baseline 2
 - 2.1 逐级量化分解 2
 - 2.2 INT8 GEMM 与 FP64 重组 2
 - 2.3 Baseline 实测 3
- 3 初始诊断 3
- 4 优化过程 4
 - 4.1 r3 已含的优化（既有状态） 4
 - 4.2 本轮优化：消除非 GEMM 固定开销 4
 - 4.3 GEMM 配置扫描 5
 - 4.4 验证与分析 5
- 5 失败尝试 5
 - 5.1 多 stream 并发（已回退） 5
 - 5.2 N 轴 GEMM 合并（已回退） 6
 - 5.3 更深 GEMM 流水（已回退） 6
- 6 最终结果对比 6
- 7 最终评分 6
- 8 总结 7

1 实验目标

本实验要求使用 INT8 Tensor Core 模拟 FP64 矩阵乘法。我们把 FP64 输入逐级分解为多个 INT8 分量，调用 INT8 GEMM 计算分量之间的部分积，再以 FP64 比例尺重组输出。在保持 L2 相对误差满足评测正确性门槛的前提下，优化 `gemm_my_int8_fp64` 的端到端吞吐量。

评测环境 (OJ config): NVIDIA H800 PCIe MIG 1g.10gb (sm_90a, 10 GB HBM3), CUDA Toolkit 13.3, 4 CPU、16 GiB 内存, 编译目标 sm_90a。测试矩阵为 4096^3 与 8192^3 , 元素为 $[-1, 1]$ 均匀分布随机数。OJ 构建环境仅提供 CUDA/cuBLAS 头文件与 `-Iinclude` (无 CUTLASS), 因此提交版 `my_int8_fp64.cu` 为自包含实现: INT8 GEMM 由手写 `mma.sync.m16n8k32` 内核完成, 不依赖任何第三方库。

2 FP64 模拟原理与 Baseline

2.1 逐级量化分解

FP64 具有 53 bit 有效尾数, 而一个 INT8 分量只能表达有限的整数范围。对 `split` 数 S , 逐级量化将矩阵元素 x 近似写成

$$x \approx \sum_{i=0}^{S-1} q_i s_i, \quad q_i \in [-127, 127] \cap \mathbb{Z} \quad (1)$$

其中 q_i 是第 i 级 INT8 分量, s_i 是全矩阵共享的 FP64 比例尺。令 $r_0 = x$, 初始比例尺由矩阵最大绝对值 $X_{\max}$ 决定, 递推关系为

$$s_0 = \frac{X_{\max}}{127}, \quad q_i = \text{round}\left(\frac{r_i}{s_i}\right), \quad (2)$$

$$r_{i+1} = r_i - q_i s_i, \quad s_{i+1} = \frac{s_i}{254} \quad (3)$$

每一级从前一级残差中提取新的有效信息。比例尺按 254 缩小, 使下一层能够覆盖上一层四舍五入后不超过半个量化步长的残差, 同时保持 INT8 数值不溢出。全零矩阵需要单独处理, 以避免计算初始比例尺时除零。

2.2 INT8 GEMM 与 FP64 重组

设矩阵 A 和 B 分别被分解为 S 个 INT8 矩阵 A_q^i 和 B_q^j , 其对应比例尺为 s_i^A 和 s_j^B 。模拟结果可写为

$$\tilde{C} = \sum_{i=0}^{S-1} \sum_{j=0}^{S-1} s_i^A s_j^B (A_q^i B_q^j) \quad (4)$$

每个括号内的乘法使用 $\text{INT8} \times \text{INT8} \rightarrow \text{INT32}$ GEMM。由于 A 与 B 各有 S 个分量, 必须组合全部 $S \times S$ 个分量对, 因此共执行 S^2 次 INT8 GEMM。各 INT32 部分积随后乘以 FP64 缩放系数, 并在 FP64 下累加至输出矩阵 C 。

整个流程可以概括为: 分别求 A 、 B 的最大绝对值, 逐级产生 INT8 分量, 执行若干次 INT8 GEMM (裁剪后为 S^2 的子集), 最后完成 FP64 缩放与重组。

2.3 Baseline 实测

在 H800 MIG 上对三个参考实现（cuBLAS FP64、朴素 INT8 模拟 int8_cublas_baseline、cuBLAS 内置 FP64 定点仿真 cublas_emulated）实测如下。

规模	方法	splits	时间 / ms	GFLOPS	最大绝对误差	L2 相对误差
4096 ³	fp64_cublas	-	1400.799	98.11	0	0
4096 ³	int8_cublas_baseline	2	70.947	1937.2	2.573e-03	2.192e-05
4096 ³	int8_cublas_baseline	4	233.104	589.6	4.345e-08	3.397e-10
4096 ³	int8_cublas_baseline	6	495.163	277.6	7.958e-13	5.685e-15
4096 ³	int8_cublas_baseline	8	862.414	159.4	6.395e-13	2.150e-15
4096 ³	cublas_emulated	2	13.227	10390.9	6.621e-05	5.395e-07
4096 ³	cublas_emulated	8	41.834	3285.4	6.253e-13	2.140e-15
8192 ³	fp64_cublas	-	11196.419	98.20	0	0
8192 ³	int8_cublas_baseline	2	448.712	2450.4	3.568e-03	2.192e-05
8192 ³	int8_cublas_baseline	4	1606.265	684.5	5.935e-08	3.398e-10
8192 ³	int8_cublas_baseline	6	3501.499	314.0	1.634e-12	6.075e-15
8192 ³	int8_cublas_baseline	8	6119.972	179.7	1.506e-12	3.026e-15

Table 1: 参考实现实测（H800 PCIe MIG 1g.10gb, CUDA 13.3, iters=5）

int8_cublas_baseline 随 splits 增大按 S^2 退化，splits=8 时 8192³ 需 6.1 秒；cublas_emulated 是 cuBLAS 13 自带的固定点仿真，精度与吞吐都强于朴素 baseline，但不参与提交评分，仅作参考对照。

3 初始诊断

以第一版提交（以下称 r3，即 OJ 实测 66 分的版本）为起点做 Nsight Systems 时间线与逐阶段事件计时。4096³、splits=4 的端到端时间约 60.4 ms，阶段分解如下。

阶段	时间 / ms	占比	说明
maxabs 归约	1.68	2.8%	两级归约 + device 侧 scale 计算
量化	10.76	17.7%	A 走 smem 分块转置, B 直写
INT8 GEMM	39.60	65.2%	13 次 <code>mma.sync</code> GEMM, 单次约 3.05 ms
FP64 重组	8.68	14.3%	批量重组, 每批读 INT32 workspace 并读改写 C

Table 2: r3 在 4096³、splits=4 的阶段分布（13 对 GEMM, 事件计时 + nsys 交叉验证）

关键观察：在 4096³、splits=2（3 对 GEMM）下，端到端 18.8 ms 中 GEMM 仅占 49%，量化、重组与归约等非 GEMM 固定开销合计占 51%，是评分权重最高（40%）的检查点上的主要瓶颈；splits>=4 时 GEMM 占比升至 65% 至 85%，但非 GEMM 的量化与重组仍有约 1 至 2 倍于带宽下限的冗余。

4 优化过程

4.1 r3 已含的优化（既有状态）

r3 提交版相对课程骨架已包含以下优化，实测正确性与性能如下。

逐级量化融合。同一矩阵的全部级别合并为一次 kernel launch，每个线程在寄存器中连续完成所有级别的量化与残差更新；A 矩阵因列主序存储需要转置为行主序，采用共享内存分块转置，并把 FP64 除法换成预计算的 $\frac{1}{s}$ 乘法。

批量 FP64 重组。连续若干 split pair 的 INT32 结果先写入一块 workspace，再由一个 kernel 批量完成多对比例尺乘法与 FP64 累加，减少 C 的读改写次数与 launch 数（kPairBatch=4）。

全异步 maxabs 与 CUDA Graph。maxabs 用两级 device 归约并把 scale 计算搬到 device，去掉 host 往返；再用 CUDA Graph 把固定形状的整条 kernel 序列捕获后重放，摊薄 launch 与初始化开销。

反对角线裁剪分级收紧。组合比例尺 $s_i^A s_j^B$ 按 $254^{-(i+j)}$ 随反对角线衰减。OJ 正确性门槛比课程文档中的 $1e-5$ 更严：曾用 $i+j < 3$ （6 对，4096³ 的 splits=4 下 L2 误差约 1.7×10^{-7} ）提交未通过，因此按 splits 分级设置裁剪半径，使误差与官方 baseline 同量级：

splits	裁剪半径 d	保留 pair 数	量化层数	L2 相对误差
2	2	3	2	2.684e-05
4	5	13	4	3.397e-10
6	7	26	6	5.685e-15
8	7	28	7	2.150e-15

Table 3: r3 的裁剪参数与实测误差（4096³；8192³ 同参数, 误差同量级）

4.2 本轮优化：消除非 GEMM 固定开销

针对初始诊断中最突出的量化与重组开销，本轮做四项修改，全部保持数值结果逐位一致。

量化舍入 FP32 化。量化商 $q_i = \text{round}\left(\frac{x}{s_i}\right)$ 改用 FP32 rintf，乘积 $\frac{x}{s_i} \in [-127, 127]$ 在 FP32 表示范围内可精确取整；残差仍以 FP64 递推。FP32 舍入的误差远低于二级量化本身的误差下限，实测各 splits 的 L2 误差逐位不变。

量化向量化。B 量化改用 double4 读入、4 路并行量化并打包为单个 uint32 写回，线程数与访存事务数降为四分之一。A 转置量化重构为单遍流水：加载阶段把列主序 tile 转置写入带 padding 的共享内存（消除写 bank 冲突），计算阶段按行主序所有权逐元素持有寄存器残差链，一次遍历完成全部级别，去掉了原实现中每级两次同步与中间 sm_out 缓冲。

重组向量化与免清零。recombine_batch_kernel 改用 int4/double4 每线程处理 4 元素；首批次以纯写模式覆盖 C （跳过读回），从而删除每次调用的 cudaMemsetAsync(dC)；后续批次仍为读改写。

重组批次扩容。kPairBatch 由 4 提升到 16（按剩余显存动态上限保护）， C 的读改写次数进一步下降。

4.3 GEMM 配置扫描

对 `mma.sync` 内核做流水级数与 `tile` 配置扫描，确认当前配置无低风险增益。

配置	4096 ³ 单 GEMM / ms	TFLOPS
STAGES=3	3.146	43.7
STAGES=4 (现状)	3.060	44.9
STAGES=5	3.141	43.8
STAGES=6	3.143	43.7
WARPS=16	3.233	42.5
CTA 128x128x64	4.231	32.5
CTA 256x128x64	4.472	30.7
CTA_K=32	3.981	34.5

Table 4: GEMM 配置扫描 (4096³; 8192³ 下 STAGES=4 为 46.6 TFLOPS, 其余更低)

STAGES=4、CTA 128x256x64、8 warp 已是最优组合，加深流水、扩 warp 或改 tile 均无收益。

4.4 验证与分析

`compute-sanitizer --tool memcheck` 零错误；全部 8 个（规模, splits）组合的 L2 相对误差与 r3 逐位一致。由于共享 MIG 切片在不同作业间有约 15% 至 25% 的时钟与邻居扰动，所有性能对比均在同一个作业内交替编译 r3 与新版、交替测量（A/B 对照）。

规模	splits	r3 / ms	新版 / ms	加速比
4096 ³	2	18.75	14.86	1.261x
4096 ³	4	60.42	50.74	1.191x
4096 ³	6	112.07	98.36	1.139x
4096 ³	8	121.27	105.12	1.154x
8192 ³	2	108.14	92.79	1.165x
8192 ³	4	387.66	354.37	1.094x
8192 ³	6	747.93	697.54	1.072x
8192 ³	8	801.71	757.58	1.058x

Table 5: 同作业 A/B 实测：r3 与新版（iters=5/4096³, iters=3/8192³）

新版在 4096³ 上提升 14% 至 26%，在 8192³ 上提升 6% 至 17%。4096³、splits=4 的阶段分布更新为：GEMM 39.60 ms 不变，量化由 10.76 ms 降至 5.26 ms（A 转置 2.66 + B 直写 2.60），重组由 8.68 ms 降至 4.35 ms，maxabs 1.65 ms，非 GEMM 合计从 21.1 ms 减半到 11.3 ms。

5 失败尝试

5.1 多 stream 并发（已回退）

A/B 量化之间没有数据依赖，重组是内存瓶颈、GEMM 是计算瓶颈，理论上存在重叠机会。实际仅并发量化的版本与串行几乎无差别；进一步对 GEMM 与重组做双缓冲流水线反而

退化，原因是 MIG 1g.10gb 只有 14 个 SM，GEMM 已吃掉大部分 SM 吞吐，第二条 stream 上的重组 kernel 与其争抢调度。予以回退。

5.2 N 轴 GEMM 合并（已回退）

尝试沿 N 轴拼接 B 分量，把多次小 GEMM 合并成少数大 GEMM。结果反而更慢：GEMM 是 FLOP-bound，合并前后总 FLOP 不变，而拼接引入的显存拷贝超过了省下的 launch 时间。予以回退。

5.3 更深 GEMM 流水（已回退）

STAGES=6（对应实验副本 my_int8_fp64_s6only）与 STAGES=3 实测单 GEMM 分别为 43.7 与 43.7 TFLOPS，均低于 STAGES=4 的 44.9，共享内存增大反而压缩了每个 SM 的调度余量。维持 STAGES=4。

6 最终结果对比

r3 为 OJ 已实测版本；新版为本次优化后的提交版（~/lab4p5/my_int8_fp64.cu，本机 H800 A/B 实测，待 OJ 复测）。

规模	splits	r3 (OJ) / ms	r3 GFLOPS	新版 / ms	新版 GFLOPS
4096 ³	2	18.742	7333	14.86	9249
4096 ³	4	60.435	2274	50.74	2709
4096 ³	6	112.600	1221	98.36	1397
4096 ³	8	122.002	1127	105.12	1307
8192 ³	2	109.085	10079	92.79	11849
8192 ³	4	391.668	2807	354.37	3103
8192 ³	6	747.760	1470	697.54	1576
8192 ³	8	809.054	1359	757.58	1451

Table 6: r3 (OJ 实测) 与新版 (本机 A/B 实测, iters=5/3) 对比

相对原生 fp64_cublas，新版在 4096³ 上快约 94 倍（1400.8 ms 对 14.86 ms），在 8192³ 上快约 121 倍（11196.4 ms 对 92.79 ms）。

7 最终评分

OJ 评分按 splits 分组，检查点（GFLOPS）与权重如下。

splits	g_0 (0 分)	g_{60} (60 分)	g_{100} (100 分)	权重
2	2500	5000	10000	40%
4	750	2500	5000	20%
6	350	1500	3000	20%
8	200	1500	3000	20%

Table 7: 评分 checkpoint (GFLOPS) 与权重

r3 的 OJ 实测得分为 66/100。新版提交后 OJ 复测为 73/100 (scoreBeforeRounding=72.85)，全部 8 个组合 correct: true，误差与 r3 逐位一致。新版实测明细如下。

规模	splits	GFLOPS	时间 / ms	单项得分	加权贡献	L2 相对误差
4096 ³	2	9064.5	15.162	87.40	34.96	2.684e-05
4096 ³	4	2702.4	50.859	61.48	12.30	3.397e-10
4096 ³	6	1403.1	97.953	54.94	10.99	5.685e-15
4096 ³	8	1305.4	105.284	51.02	10.20	2.150e-15
8192 ³	2	11894.6	92.438	100.00	40.00	2.685e-05
8192 ³	4	3142.1	349.926	65.40	13.08	3.398e-10
8192 ³	6	1603.8	685.561	61.26	12.25	6.075e-15
8192 ³	8	1490.5	737.657	59.56	11.91	3.026e-15

Table 8: 新版 OJ 实测评分明细（总分 72.85，四舍五入 73；r3 为 65.80/66）

对比 r3（66 分）与新版（73 分）：4096³ splits=2 由 71.65 升至 87.40，其余检查点亦普遍提升。注意 OJ 高分段的实际得分曲线比线性插值更平缓（如 4096³ splits=2 的 9064 GFLOPS 对应 87.40 分，线性插值为 92.5 分），后续优化应以时间（而非线性外推分数）为准。

8 总结

本实验用 INT8 Tensor Core 模拟 FP64 GEMM。提交版 my_int8_fp64 基于手写 mma.sync.m16n8k32 内核（128×256×64 主 tile、8 warp、STAGES=4 的 cp.async 流水，单 GEMM 约 45 至 47 TFLOPS），仅依赖 CUDA 与 cuBLAS，可在 OJ 构建环境（-Iinclude、-lcublas -lcudart -lcuda）中直接编译。

优化工作分两条主线。第一条在 r3 中完成：逐级量化融合、共享内存分块转置、批量重组、全异步 maxabs、CUDA Graph，以及按 OJ 正确性门槛分级的反对角线裁剪，使各 splits 的 L2 误差与官方 baseline 同量级。第二条是本轮消除非 GEMM 固定开销：量化舍入 FP32 化、量化与重组的向量化、A 转置量化的单遍寄存器化重构、重组免清零直写，以及重组批次扩容。A/B 实测新版在 4096³ 提升 14% 至 26%、在 8192³ 提升 6% 至 17%，全部 8 个组合的 L2 误差与 r3 逐位一致，compute-sanitizer --tool memcheck 零错误；OJ 复测总分由 66/100 升至 73/100。

失败尝试同样有启发。多 stream 并发在 14 个 SM 的 MIG 切片上只会加剧资源竞争；N 轴 GEMM 合并 FLOP 总量不变的前提下引入额外拷贝；更深或更浅的 GEMM 流水线（STAGES=3/5/6）以及扩 warp、改 tile 均不优于现状。三者都提示：在没有把底层瓶颈真正消除之前，调度与配置层面的技巧收益有限；当前 GEMM 已接近 mma.sync 路径的实际上限，进一步的吞吐提升需要换用 wgmma 与 TMA 等 sm_90a 原生异步路径，留待后续工作。